

Решение задачи:

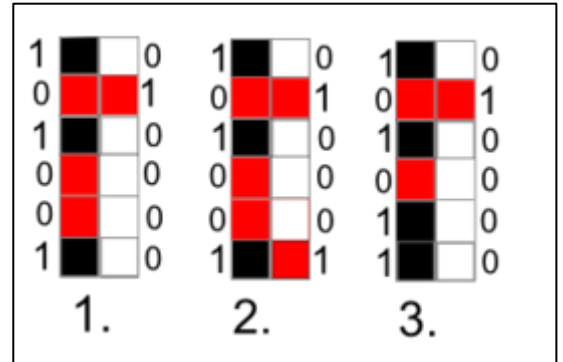
Дискретное программирование по профилю – замощение доминошками

Задание:

Найти количество способов замостить таблицу $n \times m$ с помощью доминошек размерами 1×2 , 2×1 .

Решение:

Для удобства можно хранить профили в виде двоичных масок. В качестве состояния динамики будем использовать профили размерами n . В этом профиле 1 будет означать, что домино лежит горизонтально и заканчивается на этом столбце, иначе 0. Таких профилей будет 2^n . Теперь проверим из какого профиля в какой можно перейти.



С чего мы начали:

Мы создаем функцию для подсчета количества способов:

n – число строк, m – число столбцов

```
def tiling_dominoes(n, m):  
    if (n * m % 2 != 0):  
        return "Your table cannot be paved with 2 x 1 and 1 x 2 dominoes."
```

Первое, что мы проверяем это возможно ли вообще замостить таблицу нашими доминошками. То есть наша таблица должна состоять из четного количества ячеек, иначе ее будет не заполнить.

Далее мы создаем массив d , который будет хранить информацию о том из какого профиля в какой можно перейти и для начала просто заполним его 0:

```
7 profiles = 2**n # 2^n профилей  
8 d = [[0] * profiles for i in range(profiles)]
```

Далее мы будем строить матрицу переходов:

d – содержит 2^{2n} элементов и 4 вложенных списка. Каждый список будет нам говорить в какой профиль можно будет перейти из этого. То есть для первого вложенного списка, индекс которого 0 (в двоичной системе 00 для $n = 2$) будет соответствовать тому, в какой профиль можно перейти из 00 (для $n = 2$). И так же для всех остальных. Как же мы будем это проверять?

Наши условия:

Из профиля i в профиль j можно перейти если выполняются условия:

Можно положить горизонтальные домино. То есть там, где в j профиле стоит 1, в i профиле должен стоять 0.

Можно доложить в оставшиеся клетки вертикальные домино. То есть оставшиеся 0 в i профиле должны образовывать четные подстроки. Сначала мы проверяем первое условие. После уже второе. Для проверки второго условия мы решили объединить два профиля в один. И проверяем на наше условия. Если все условия удовлетворяют, то мы ставим 1 туда, если же не так, то 0. Так теперь мы знаем из каких в какие профили возможен переход.

```

10 # Построение матрицы переходов
11 #мы делаем матрицу d такую, которая показывает нам из какого профиля можно
    перейти в какой например: если d[0] = [1,0,0,1], то это значит, что из
    0 столбца, то есть из 00 можно перейти в 00 и в 11 и больше никуда
12 for j in range(profiles):
13     for i in range(profiles):
14         # проверяет не пересекаются ли 1 в нашем и следующем столбце
            например: 01(1) и 11(3) = 01(1), и если где то будет не 00, то
            значит оно нам не подходит
15         if (j & i) != 0:
16             continue
17         combined = j | i # делает n битов, которые содержат объединение
            битов нашего столбца и следующего. Например: если нынешний = 00
            (0) а следующий = 11(3), то их объединение = 11
18         count = 0
19         valid = True
20         for bit in range(n): # проходимся по нашему combined = 11 (как в
            примере) и считаем с первого бита, то есть с 1 или если наш
            combined = 10, то с 0 и потом 1 и если это = 1, то проверяем
            на количество 0 до этой единицы и делаем его False если значит
            не равно нулю и заканчиваем этот цикл, а если кратно 2, то
            значит count = 0
21         if (combined >> bit) & 1:
22             if count % 2 != 0:
23                 valid = False
24                 break
25                 count = 0
26         else:
27             count += 1
28         if valid and (count % 2 == 0):
29             d[j][i] = 1

```

Теперь мы создадим новый массив для подсчета количества способов. Этот массив будет содержать $m+1$ списков, в каждом из которых будет количество способов добраться туда. Мы создаем $m+1$, для того чтобы добраться до столбца, который будет после – последнего и он должен быть равен 0 (00 для $n = 2$)

Теперь нам нужно как – то заполнить этот массив. Мы будем проходиться по столбцам. Первый столбец может быть только 0 (00 для $n = 2$). Далее мы будем проходиться по каждому элементу нашего списка a . И в каждый элемент будем записывать количество способов добраться до туда. Например: возьмем первый столбец (то есть второй). $a[1][i] = \text{sum}(a[k-1][i] * d[i][j] \text{ for } j \text{ in range(profiles)})$. Умножаем на элемент из d , чтобы либо обнулялось если нельзя из предыдущего добраться в наш столбец или умножаем на 1, если можно.

И после того, как мы посчитали количество способов добраться до такого – то профиля мы просто выводим количество способов добраться до $a[m][0]$, так как после – последнего столбца ничего быть не может. Это и будет наш ответ.

```

31 a = [[0] * profiles for i in range(m + 1)]
32 a[0][0] = 1 # так как первым столбец может быть только 0, в нашем случае 00
33
34 # Заполнение динамики
35 for k in range(1, m + 1): # проходимся по всем столбцам до m+1 так как мы
    еще создаем столбец после последнего, чтобы сосчитать количество
    способов прийти в 00 в конце
36 for i in range(profiles): # проходимся по всем профилям
37     a[k][i] = sum(a[k-1][j] * d[i][j] for j in range(profiles)) #
        считаем для каждого профиля количество возможных способов туда
        перейти на такой то столбец
38 return a[m][0]
39 print(tiling_dominoes(int(input()), int(input())))
40

```

Подсчитав сложность она будет равна $O(2^{2n}(m + n))$, но обычно столбцов количество преобладает и поэтому принято записывать $O(2^{2n}m)$

Оценка сложности: подсчет $d - 2^{2n}$, и подсчет $a - 2^{2n}m$ в итоге $O(2^{2n}m)$.

(Википедия)